

ASSIGNMENT 1: WEB TECHNOLOGIES FUNDAMENTALS

Course/Internship: Cyber Security Portfolio Lab

Submitted by: Rajitha Ale

TASK 1: Basic HTML Pages & Client-Side Interaction

1.1 Source Code for HTML Pages

I created 5 foundational HTML pages inside my local environment to test web layout elements, including forms, tables, and lists.

Page 1 (POST Form): `form\_post.html` (Handles secure user registration parameters).

Page 2 (GET Form): `form\_get.html` (Handles search engine queries via URL parameters).

Page 3 (Tables): `table\_data.html` (Displays system logs and operational inventory structures).

Page 4 (Lists): `lists\_info.html` (Organizes security hardening step checklists).

Page 5 (Combined Portal): `combined\_portal.html` (Integrates lists, forms, and tables into an administrative screen).

1.2 Client-Side DOM Modification

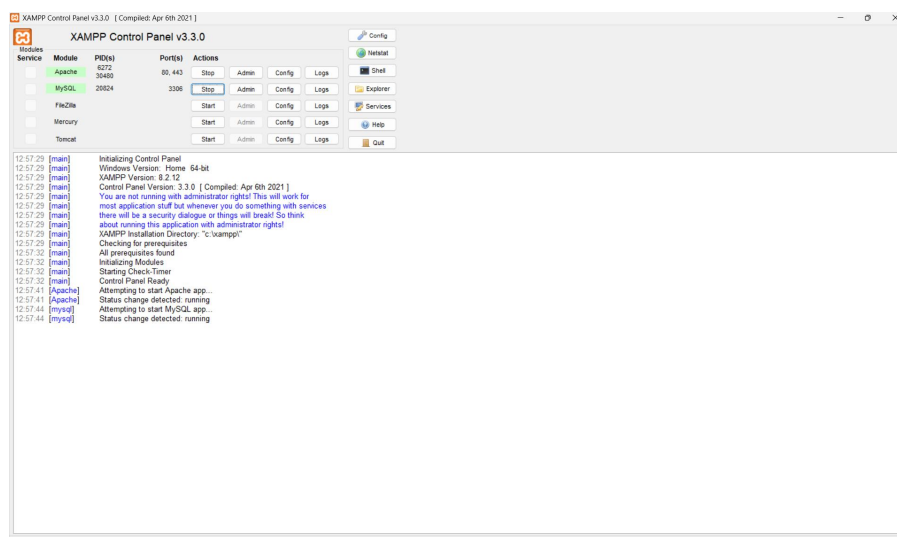
Using the browser Developer Tools Console (`F12`), I manipulated the Document Object Model (DOM) in real-time by executing the following JavaScript command on `combined\_portal.html`:

```
`document.querySelector('h2').innerText = "Hacked by Javascript Console!";`
```

TASK 2: PHP Scripting & Database Integration

2.1 Environmental Setup

I installed and configured **XAMPP** locally to handle server-side processing. Both the Apache Web Server and MySQL Database services were successfully initialized and

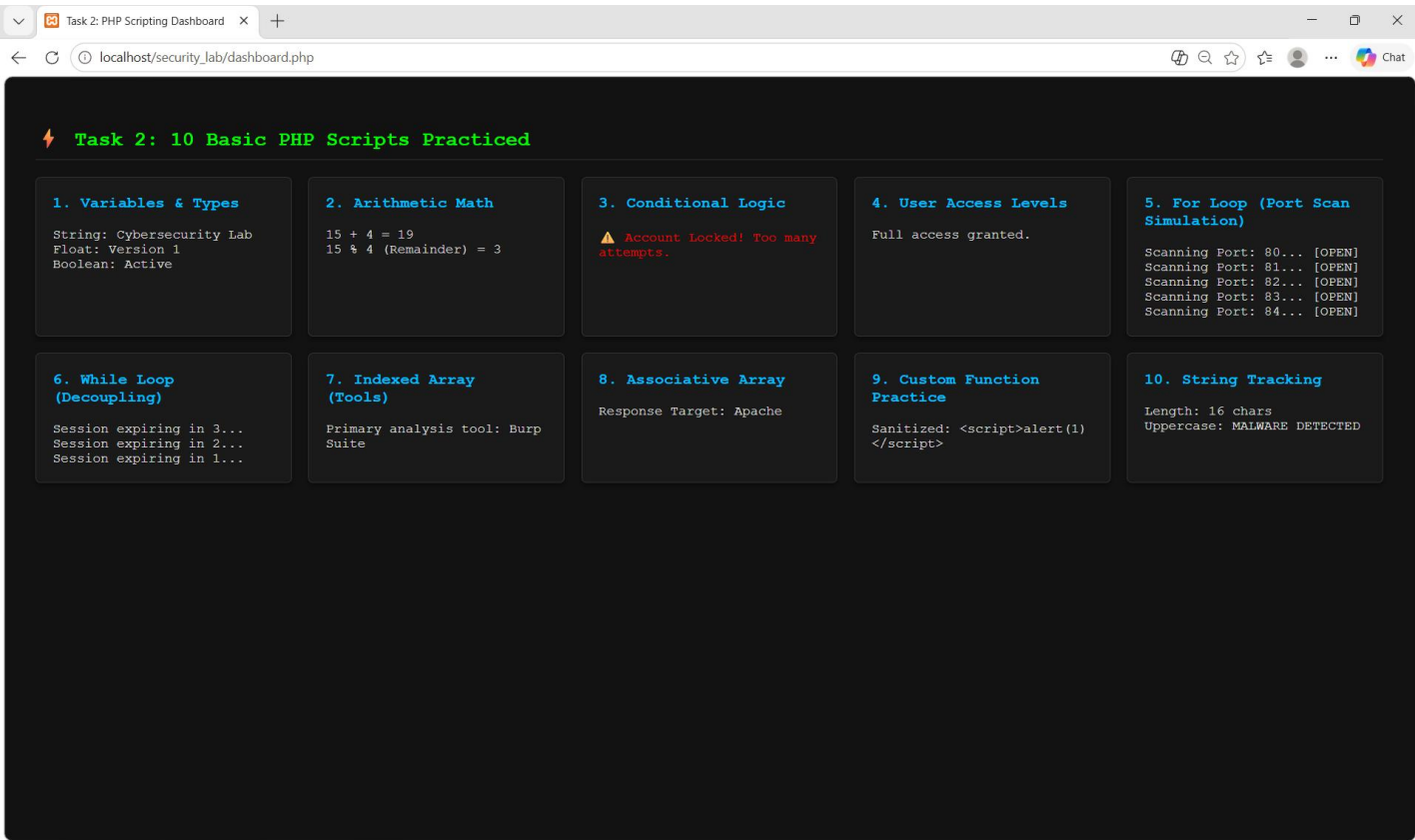


verified.

2.2 Core Scripting Elements

I designed a centralized dashboard script (`dashboard.php`) executing 10 baseline PHP behaviors:

- 1. Variables & Data Types
- 2. Arithmetic Operators
- 3. Conditional Logic (If/Else)
- 4. Switch Case Blocks
- 5. For Loops (Simulating a network port scan sequence)
- 6. While Loops
- 7. Indexed Arrays
- 8. Associative Arrays
- 9. Custom Sanitization Functions
- 10. String Manipulation



2.3 Secure Authentication Architecture

I created a dynamic database link mapping file (`db_connect.php`) using standard PHP Data Objects (PDO). A backend validation query script (`login.php`) was successfully cross-linked to verify user inputs against records stored inside the `users` table of a local MySQL database instance (`lab_db`).

TASK 3: HTTP Request/Response Analysis

3.1 HTTP Header Inspection

Using the built-in browser Developer Tools **Network Tab**, I captured the traffic generated when submitting credentials to `login.php`.

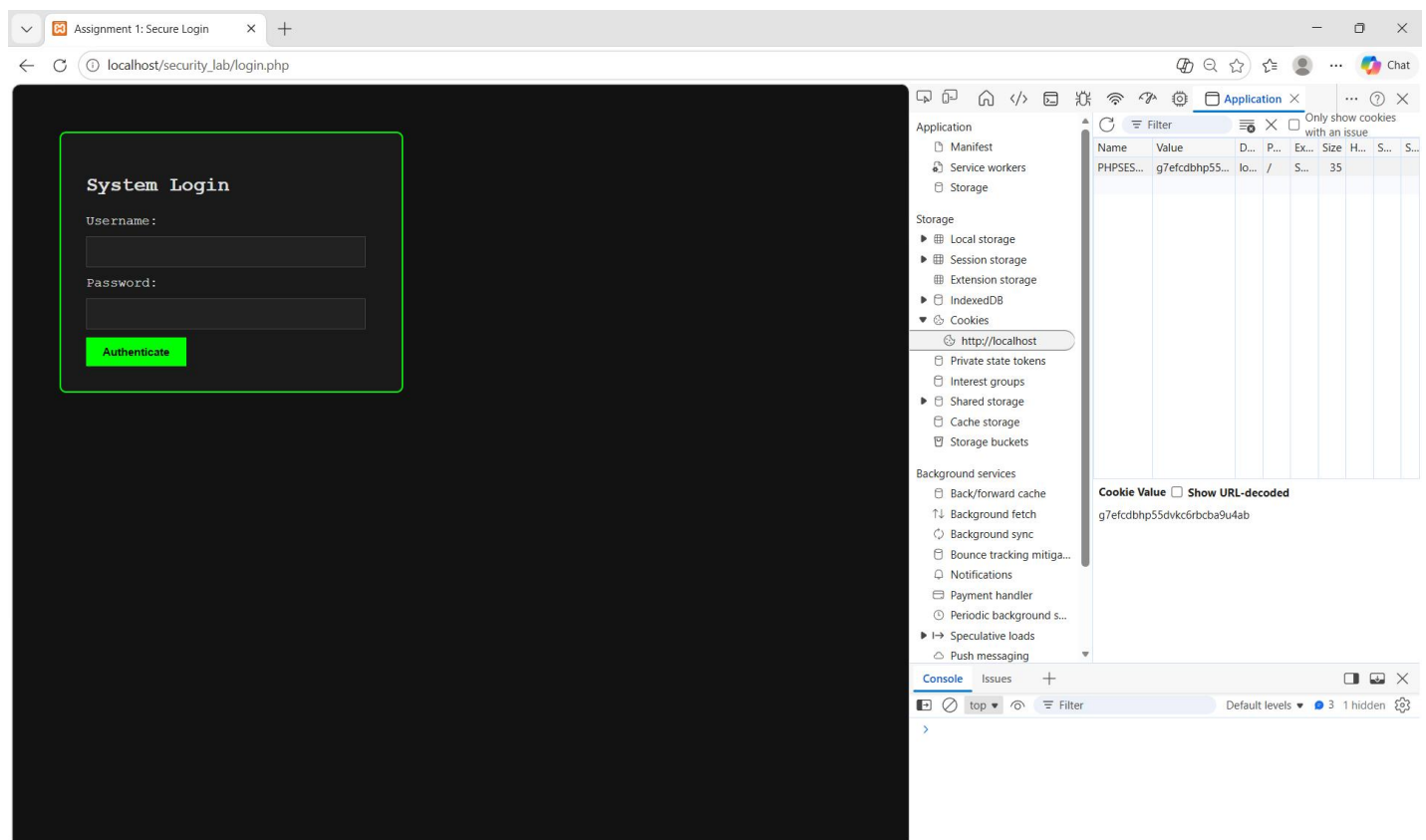
Request Method: Determined to be `POST` to hide credential payloads from the URL bar.

Content-Type Header: Identified as `application/x-www-form-urlencoded`.

User-Agent Header: Revealed the underlying client operating system, hardware configuration, and core browser architecture.

3.2 State Management and Session Tracking

By switching to the **Application** browser tab, I inspected the cookie vault for `http://localhost`. After introducing `session\_start();` to the backend source code, the server successfully transmitted a `PHPSESSID` token value to the client browser to maintain an authenticated active session state.



TASK 4: Source Code Analysis & Tech Footprints

I performed a structural search across multiple live and local index views to audit structural implementation choices.

Summary Table of Analyzed Domains:

Target Platform	Form Actions Identified	Hidden Fields Discovered	Observed Technologies (via Wappalyzer)
Local Portal	action="login.php" method="POST"	None	Apache, PHP, MySQL
Wikipedia	action="//www.wikipedia.org/search-redirect.php"	name="family" value="wikipedia"	Apache Traffic Server, Varnish
Google Home	action="/search" method="GET"	name="source" value="hp"	Google Web Server (GWS)
Stack Overflow	action="/search" method="GET"	Context state hashes	ASP.NET, IIS, MS SQL Server
GitHub	action="/search" method="GET"	CSRF Anti-Forgery Tokens	Ruby on Rails, Go, React
Internet Archive	action="/web/query" method="GET"	Deployment lifecycle labels	Apache, Ubuntu Linux
WordPress View	action="wp-comments-post.php"	name="comment_post_ID"	WordPress CMS, Nginx, PHP
XAMPP Dashboard	No active data form instances	Cross-linking markers	Apache Server, OpenSSL, PHP
DuckDuckGo	action="/" method="GET"	name="t" value="h_"	Nginx Web Server, Linux OS
OWASP Juice Shop	Managed via JS/API Endpoints	Administrative notes	Node.js, Express, Angular, SQLite

Code Repository: <https://github.com/rajithaale1502-nn/cybersecurity-portfolio-lab/upload/main>